

Scaling Guide — Adding UAVs, Nodes and Boards

What an ns-3 node and a TAP actually are, how ROS 2 reaches across two machines, and the checklist for adding **board N** — written so the same procedure applies to every board, not one of them.

1. The four layers

Every drone in this testbed exists in four places at once. Adding a UAV means adding it to all four — miss one and the failure is usually silent.

Layer	What it is	Where it lives
Gazebo model	the airframe, its camera, its FDM ports	models/iris_N_netns/
SITL	the autopilot — ArduPilot process	one process per UAV
ROS 2	topics, services, the bridge to the GCS	micro_ros_agent + drone_bridge
ns-3 node	a radio with a position in 3D space	the .cc scenario file

2. Node, TAP, bridge — what they actually are

Node — inside ns-3, a simulated computer holding a WifiNetDevice (a simulated 802.11a radio) and a mobility model (its position). All radios share one channel. When one transmits, ns-3 computes for every other node: distance, path loss, obstacle shadowing, fading, SNR — then decides whether the packet decodes. That calculation is the whole point of the simulator, and it depends entirely on position.

TAP — a virtual network interface on Linux. The kernel treats it like a real network card, but instead of a wire on the other side, packets are handed to a userspace program. Here that program is ns-3.

TapBridge — the ns-3 component that ties one TAP to one node's radio. It is the airlock: a real Ethernet frame goes in, crosses the simulated air with real loss and delay, and comes out as a real frame on the other side.

Bridge (br-uavN) — a Linux software switch. It joins the TAP to whatever should sit behind that radio: a network namespace, a physical VLAN, or both.

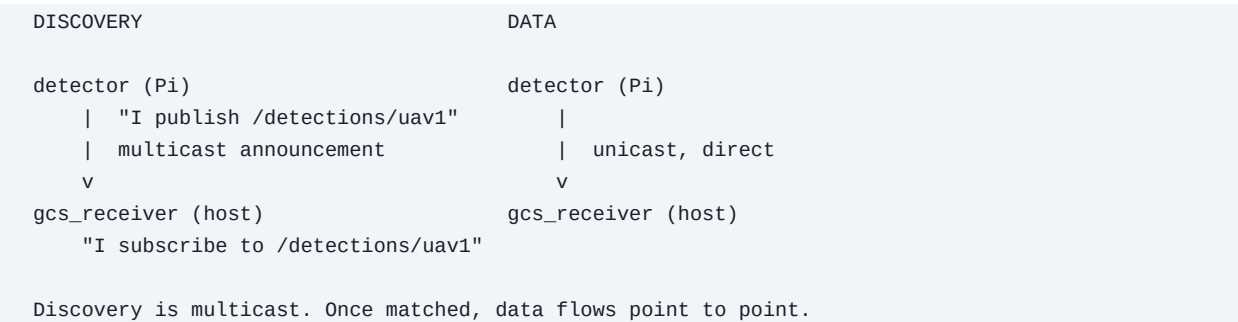
THE FULL PATH OF ONE DETECTION

```
Pi (board N): detector publishes /detections/uavN
  | real UDP packet, tagged with the radio VLAN
  v
switch --- host NIC --- eth-rf --- br-uavN --- tap-uavN      REAL LINUX
                                   |
                                   TapBridge (the airlock)
                                   |
ns-3 node N radio ===> simulated 802.11a air ===> ns-3 node 0 radio
                        distance / buildings / fading / contention
                                   |
tap-gcs --- br-gcs --- veth --- gcsns --- gcs_receiver      REAL LINUX
```

Everything outside the airlock happens on real hardware.
Everything inside it is arithmetic.

3. How ROS 2 connects across two machines

ROS 2 has no central server. Nodes find each other with **DDS discovery**: each participant announces itself by multicast, and any node interested in a matching topic connects directly. Nothing needs configuring for two nodes on one machine to find each other — which is exactly why it is easy to get wrong across two.



The interface whitelist. A machine with WiFi and Ethernet has several addresses, and Fast DDS announces a locator on every one of them. The Pi can then advertise its WiFi address, the host connects to it, and the camera stream silently takes the WiFi path — where large samples fragment and drop. `config/fastdds_hitl_eth.xml` pins DDS to the wired addresses only.

! Fast DDS 2.6 (ROS 2 Humble) reads the interface list ONCE, at participant creation, and has no dynamic interface detection. An address that appears after a process starts can never be used by it. Bring the addresses up **before** launching Gazebo.

4. The two networks, and why they are separate

Each drone has two data paths that must never be confused.

	Sensor / management	Wireless
Carries	camera frames, SSH, chrony	detections, telemetry
Subnet	10.0.N.x (VLAN 10, 11, 12)	10.42.0.x (VLAN 42, 43, 44)
Through ns-3	No — plain cable	YES — this is what is measured
Represents	the camera ribbon cable inside one airframe	the radio link
Measured	1.4 ms, 0% loss	74.5 ms, 38 ms jitter

Degrading the sensor link would be physically meaningless — on a real drone the camera is bolted to the companion computer. Routing enforces the split with no firewall rules: Gazebo lives on 10.0.N.1 and can only reach the Pi at 10.0.N.2; gcsns lives on 10.42.0.10 and can only reach it at 10.42.0.1x.

! Give every board its own VLAN ids. Two Pis sharing a VLAN can reach each other directly through the switch at 1.4 ms, bypassing ns-3 entirely — and every UAV-to-UAV latency number becomes meaningless with no error anywhere.

5. Adding one more UAV (software only)

Every UAV needs its own ports at four levels. The numbers below are the existing convention — follow it and nothing collides.

	UAV 1	UAV 2	UAV 3
MAVLink TCP	5760	5770	5780
FDM in / out	9002 / 9003	9012 / 9013	9022 / 9023
DDS UDP port	2019	2020	2021
FDM address	172.31.1.1	172.31.2.1	172.31.3.1
ns-3 node / tap	1 / tap-uav1	2 / tap-uav2	3 / tap-uav3
Radio address	10.42.0.11	10.42.0.12	10.42.0.13

Step by step

1. Gazebo model. models/iris_N_netns/ already exists for N = 1,2,3 with the correct FDM ports and camera topic. Adding a fourth means copying the directory and changing fdm_port_in, listen_addr and the camera topic name.

2. Add it to the world. The netns world currently includes only iris_1_netns. A multi-UAV world includes each model with its own pose.

3. DDS parameters. Copy params/uav1_dds_netns.parm and change DDS_UDP_PORT. It points the autopilot at the micro_ros_agent in gcsns.

4. SITL instance — one process per UAV, each in its own namespace:

```
sudo ip netns exec uavNns strace -f -e trace=none -o /dev/null \  
    <ardupilot>/build/sitl/bin/arducopter \  
    --model gazebo-iris --sysid N --sim-address=172.31.N.1 -I<N-1> \  
    --defaults params/uavN_dds_netns.parm  
  
# strace is NOT for logging. Being ptrace-traced suppresses SITL's  
# auto-reboot loop, which otherwise restarts ~25 times in 15 s when DDS  
# is enabled inside a namespace. The reboot preserves the PID, so a  
# liveness check reports "alive" throughout and sees nothing wrong.
```

5. ROS 2 side — one micro_ros_agent and one drone_bridge per UAV, both inside gcsns:

```
ros2 run micro_ros_agent micro_ros_agent udp4 --port 2020  
ros2 run uav_controller drone_bridge --ros-args -p uav_id:=2 -p mavlink_port:=5770
```

6. ns-3 node — a TAP and a bridge, created by the launcher:

```
ip tuntap add dev tap-uavN mode tap user $USER  
ip link add name br-uavN type bridge  
ip link set tap-uavN master br-uavN  
ip link set br-uavN up  
# then attach whatever sits behind that radio:  
#   a namespace -> veth pair into uavNns  
#   a Pi        -> the physical VLAN leg (eth-rf.4X)
```

6. Adding board N (hardware node)

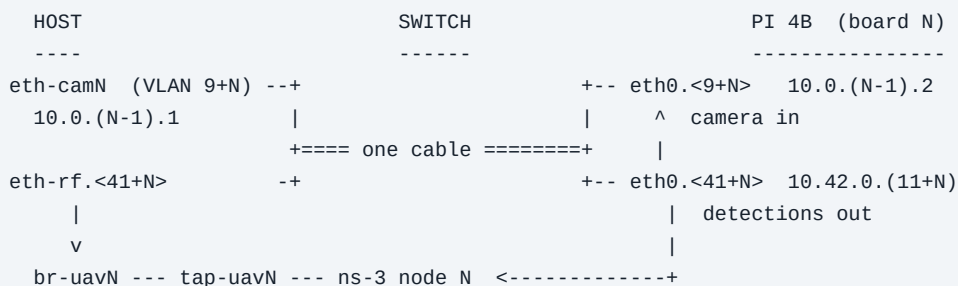
A Pi replaces the software namespace with real silicon. Everything below is written for **board N** — substitute N and the same steps build any board.

The addressing formula

BOARD N	N = 1, 2, 3, ...	
hostname	uav-pi-0N	
sensor VLAN id	9 + N	-> 10, 11, 12
sensor addresses	host 10.0.(N-1).1 / Pi 10.0.(N-1).2	
radio VLAN id	41 + N	-> 42, 43, 44
radio address	10.42.0.(11 + N)	-> .12, .13, .14
ns-3 node / tap	N / tap-uavN	

Board	Sensor VLAN	Pi address	Host address	Radio VLAN	Radio address
uav-pi-01	10	10.0.0.2	10.0.0.1	42	10.42.0.12
uav-pi-02	11	10.0.1.2	10.0.1.1	43	10.42.0.13
uav-pi-03	12	10.0.2.2	10.0.2.1	44	10.42.0.14

ONE CABLE, TWO LOGICAL LINKS, PER BOARD



The switch need not understand VLANs — tagging happens at the endpoints, so it only has to forward tagged frames, which any gigabit switch does.

Host side — GUI

Network Connections → + → VLAN → Create. Connection name and VLAN interface name = **eth-camN**. Parent = the USB adapter *selected by MAC*, not "via Wired connection". VLAN id = **9+N**. Then IPv4 Settings: Manual, address **10.0.(N-1).1 / 24**, gateway empty, and tick *Use this connection only for resources on its network*. IPv6 Settings: Disabled.

Pi side — netplan

```
# /etc/netplan/60-hitl-vlans.yaml  chmod 600, root:root.  Shown for N=2.
network:
  version: 2
  ethernets:
    eth0: {dhcp4: no, optional: true}
  vlans:
    eth0.11: {id: 11, link: eth0, addresses: [10.0.1.2/24]}
    eth0.43: {id: 43, link: eth0, addresses: [10.42.0.13/24]}

sudo netplan generate  # parses only – this is the syntax check
sudo netplan try       # applies, auto-reverts after 120 s
```

! Only ONE netplan file may define eth0. Two files giving the same address to different interfaces both take effect — netplan merges without warning. Ping still works, multicast silently never arrives, and Fast DDS segfaults at participant creation with no message at all. Check with: `grep -l eth0 /etc/netplan/*.yaml`

7. Limits you will hit

Limit	Value	Why
ns-3 nodes	4 (GCS + 3)	hardcoded array in the .cc — one TAP name each
Machines per node	1	TapBridge UseLocal tracks a single MAC address
Switch ports	5 (LS1005G)	host + 4 boards
Camera bandwidth	29.5 Mbps per Pi	640x384 RGB at 5 Hz, plus 5% overhead
Host uplink	~940 Mbps	measured — about 30 Pis before saturation

The node budget is the real constraint. Four nodes means GCS plus three radios. Giving SITL and its Pi separate nodes spends two on one airframe — which also models one aircraft as two contending radios, an artifact that does not exist in reality.

Three ways past it

a. Route through the namespace. Put the Pi behind uavNns rather than beside it — uavNns gets a second interface and IP forwarding, and the Pi uses it as gateway. One MAC on the channel, so UseLocal is satisfied and one node equals one aircraft. Cost: DDS multicast does not cross a router, so unicast discovery peers become necessary.

b. Patch AdhocWifiMac::SupportsSendFrom() to return true, then use TapBridge UseBridge and put both behind one bridge. Mechanically sound — ad-hoc frames carry the source in addr2 — but it is a simulator modification that has to be disclosed.

c. Extend the scenario past four nodes. The tapNames array and its loops become N-parameterised. The largest change, and the right one if the swarm grows.

8. Verification checklist

Check	Command	Expected
One address, one interface	<code>ip -4 addr show grep -c 10.0.(N-1).2/24</code>	1, never 2
Sensor link	<code>ping -c 30 10.0.(N-1).1</code>	~1.4 ms, 0% loss
Throughput	<code>iperf3 -c 10.0.(N-1).2 -t 10</code>	~940 Mbps, 0 retr
Clock	<code>chronyc tracking grep 'System time'</code>	a few microseconds
Socket buffers	<code>sysctl net.core.rmem_max net.core.wmem_max</code>	both 536870912
ns-3 sees the node	<code>grep 'missing node IDs' /tmp/ns3_single.log</code>	your node absent
Node is moving	<code>grep '^t=' /tmp/ns3_single.log</code>	coordinates change
ISOLATION	from board M: <code>ping -c 30 10.42.0.(11+N)</code>	~75 ms, NOT 1.4 ms

! The isolation check is the one that matters most. If two Pis answer each other in 1.4 ms, they are talking through the switch and skipping ns-3 — any two boards. It is the same class of fault as the shared-memory bypass detections without crossing the simulated radio at all. Nothing errors; the numbers simply describe a link that was never simulated.

OK Use at least 30 pings on the radio path. The first packet carries an ARP exchange across the simulated channel, and a 2-ping sample once reported 167 ms for a link that measures ~75 ms over 30.